

Janus 5.0 — Canonical Specification

Version 0.1 · 2025-07-01

 **Audio Overview:** *(record and drop your link here once produced)*

Table of Contents

1. [Introduction](#)
2. [Core Variables & Data Schema](#)
 1. [Node-Level Variables](#)
 2. [Edge-Level Variables](#)
 3. [Global Graph Variables](#)
 4. [JSON Examples](#)
 5. [Psychological Interpretation Notes](#)
3. [Derived Metrics & Formulas](#)
4. [Core Functions & Algorithms](#)
5. [Parameters & Thresholds](#)
6. [Data Structure & JSON Standards](#)
7. [Support Utilities](#)
8. [Future Expansion Modules](#)
9. [Glossary of Terms](#)
10. [Changelog & Version Notes](#)

1 Introduction

Janus 5.0 is a cognitive-recursion engine that represents a psyche as a **directed, weighted graph**. This specification consolidates every mathematical definition, data structure, algorithm, and safety parameter required to implement or audit the system.

Scope: Internal psychological modeling only. Interfaces, hardware hooks, and speculative future tech are **out of scope**.

2 Core Variables & Data Schema

2.1 Node-Level Variables

Field	Type	Range	Description
id	string	—	Unique identifier (e.g. <code>node_001</code>)
type	enum(belief,memory,schema,emotion,contradiction)	—	Cognitive category
base_weight	float	0 – 1	Importance/persistence under recursion
stability_score	float	0 – 1	Resistance to change
recursion_depth	int	0 +	Reflective nesting depth
load_pressure	float	0 – 1	Tension from contradictions + emotions
labels	list	optional	Semantic or experimental tags

2.2 Edge-Level Variables

Field	Type	Range	Description
source	string	—	Start node id
target	string	—	End node id
weight	float	0 – 1	Strength of influence
type	enum(reinforce,contradict,blend,anchors_on,projects_to)	—	Nature of relationship
timestamps	list	optional	Historical modification log

2.3 Global Graph Variables

Variable	Type	Range	Meaning
global_stability_index	float	0 – 1	Resilience of core schema
average_contradiction_density	float	0 – 1	System-wide conflict saturation
overall_projection_bias	float	0 – 1	Ratio of forward-projection vs memory anchoring
average_recursion_depth	float	0 +	Mean reflective complexity
system_entropy	float	≥ 0	Disorder/risk metric

2.4 JSON Examples

```
// Node example
{
  "id": "node_001",
  "type": "belief",
  "base_weight": 0.75,
```

```
"stability_score": 0.90,
"recursion_depth": 3,
"load_pressure": 0.22,
"labels": ["self-esteem", "career"]
}

// Edge example
{
  "source": "node_001",
  "target": "node_017",
  "weight": 0.6,
  "type": "contradict"
}
```

2.5 Psychological Interpretation Notes

- **base_weight** ≈ emotional/historical persistence.
- **stability_score** ≈ resistance to modification under stress tests.
- **recursion_depth** gauges introspective layering.
- **load_pressure** flags nodes at risk of collapse when overloaded.

3 Derived Metrics & Formulas

All formulas assume normalized edge weights unless otherwise noted.

Metric	Formula (Markdown LaTeX)	Output Range	Interpretation
Contradiction Density	$CD(C)=\frac{\sum_{e\in C,;t_e=\text{contradict}}w_e}{\sum_{e\in C}w_e}$	0 – 1	Local conflict saturation
Cognitive Load Pressure	$LP(n)=\sum_{e\in\text{in}(n),;t_e\in\{\text{contradict},\text{emotion}\}}$	w_e	\$

Coherence Mass	$CM(S)=\frac{1}{H(S)+\varepsilon}$ where H is entropy	> 0	Structural stability
Projection Bias	$PB(G)=\frac{\sum_{e \in E, t_e=\text{projects_to}} w_e}{\sum_{e \in E} w_e}$	$0 - 1$	Future-simulation vs memory anchoring
Graph Entropy	$H(S)=-\sum_i p_i \log p_i$	≥ 0	Disorder/uncertainty
Global Stability Index	$GSI=\text{mean}(\text{stability_score of core schema nodes})$	$0 - 1$	Resilience snapshot
Average Recursion Depth	$ARD=\text{mean}(\text{recursion_depth})$	$0 +$	Introspective complexity

4 Core Functions & Algorithms

4.1 Calculation Functions

```
calculate_contradiction_density(subgraph)
compute_load_pressure(node)
calculate_coherence_mass(subgraph)
calculate_projection_bias(graph)
graph_entropy(subgraph)
```

4.2 Transformation & Experiment Functions

```
apply_contradiction_injection(nodes, intensity)
normalize_weights(subgraph)
```

4.3 Monitoring & Safety Functions

```
recursive_consistency_check(depth)
flag_instability()
```

4.4 Snapshot & Rollback Utilities

snapshot_state()
rollback_to_snapshot(snapshot)

5 Parameters & Thresholds

5.1 System Parameters

Parameter	Type	Default	Purpose
max_recursion_depth	int	10	Limits introspective nesting
paradox_tolerance	float	0.35	Max average contradiction density before containment
load_pressure_threshold	float	0.65	Node strain trigger point
entropy_collapse_cutoff	float	0.25	Minimum acceptable coherence mass
projection_bias_alert	float	0.8	Flags over-simulation
rollback_on_flag	bool	true	Auto-revert on critical breach
epsilon_stabilizer	float	0.001	Prevents divide-by-zero in CM

5.2 Sample Parameter Block

```
{
  "max_recursion_depth": 10,
  "paradox_tolerance": 0.35,
  "load_pressure_threshold": 0.65,
  "entropy_collapse_cutoff": 0.25,
  "projection_bias_alert": 0.8,
  "rollback_on_flag": true,
  "epsilon_stabilizer": 0.001
}
```

6 Data Structure & JSON Standards

- **Node Schema** and **Edge Schema** are shown in §2.4.
- **Graph Snapshot** structure:

```
{  
  "nodes": [...],  
  "edges": [...],  
  "global_metrics": {  
    "global_stability_index": 0.88,  
    "average_contradiction_density": 0.12,  
    "overall_projection_bias": 0.33,  
    "average_recursion_depth": 2.7,  
    "system_entropy": 1.42  
  },  
  "parameters": { ... },  
  "timestamp": "2025-07-01T14:23:45Z"  
}
```

- **Change-Log Convention:** Every mutative operation logs (`before_snapshot`, `after_snapshot`, `operation_name`) for full auditability.

7 Support Utilities

- `audit_change_log(before, after, operation)`
- `validate_graph_integrity()`
- `compare_snapshots(snapshot1, snapshot2)`

8 Future Expansion Modules

- 1. **Multi-Path Scenario Forecast** — parallel perturbation paths.
- 2. **Core Schema Identifier** — central-belief detection.
- 3. **Ambiguity Saturation Test** — resilience under uncertainty.
- 4. **Entropy Gradient Analyzer** — maps local entropy growth.
- 5. **Simulated Recovery Protocols** — post-collapse stabilizers.
- 6. **Meta-Experiment Auditor** — randomized resilience trials.

9 Glossary of Terms

Term	Definition
stability_score	Node resistance to modification
coherence_mass	Inverse entropy — structural stability
contradiction_density	Ratio of conflicting to total edge weight
projection_bias	Forward-simulation vs memory emphasis
load_pressure	Aggregate strain on a node
recursion_depth	Depth of reflective chain

10 Changelog & Version Notes

Version	Date	Notes
0.1	2025-07-01	Initial canonical specification consolidated

11 License

Released under the MIT License. See [LICENSE](#) file for details.

```
"""graph_engine_stub.py
```

```
-----  
Janus 5.0 Core Graph Engine — Stub Implementation
```

```
Spec Version: 0.1 (2025-07-01)
```

```
Language : Python 3.12 + NetworkX ≥ 3.0  
-----
```

```
This file boots a minimal viable Graph Engine that:
```

```
* Enforces the canonical Node & Edge schemas (§2).
```

```
* Calculates derived metrics (§3).
```

```
* Applies safety thresholds (§5).
```

```
* Supports snapshot/rollback utilities.
```

```
* Leaves TODO hooks for future modules.
```

```
  
NOTE: All values & formulas mirror janus_5_spec.md. ⚠ Keep this  
file and the spec in lock-step—update version header on either change.
```

```
"""
```

```
from __future__ import annotations
```

```
  
from dataclasses import dataclass, asdict
```

```
from typing import List, Dict, Any
```

```
from datetime import datetime
```

```
import json
```

```
import networkx as nx
```

```
  
# =====
```

```
# Dataclass Definitions
```

```
# =====
```

```
@dataclass
```

```
class NodeData:
```

```
    id: str
```

```
    type: str # belief|memory|schema|emotion|contradiction
```

```
    base_weight: float
```

```
    stability_score: float
```

```
    recursion_depth: int
```

```
    load_pressure: float
```

```
    labels: List[str] | None = None
```

```
  
@dataclass
```

```
class EdgeData:
```

```
    source: str
```

```
    target: str
```

```
    weight: float
```

```
    type: str # reinforce|contradict|blend|anchor|reflects_on|projects_to
```

```
    timestamps: List[str] | None = None
```

```

# DOG/GOD annotation - optional meta-alignment tag
alignment: str | None = None # DOG | GOD | None

# =====
# Engine Core
# =====
class JanusGraphEngine:
    """Wraps an nx.DiGraph with Janus 5.0 cognitive-safety plumbing."""

    def __init__(self, parameters: Dict[str, Any] | None = None):
        self.graph = nx.DiGraph()
        self.snapshots: List[Dict[str, Any]] = []
        # Default parameters from §5 if none provided.
        self.params = parameters or {
            "max_recursion_depth": 10,
            "paradox_tolerance": 0.35,
            "load_pressure_threshold": 0.65,
            "entropy_collapse_cutoff": 0.25,
            "projection_bias_alert": 0.8,
            "rollback_on_flag": True,
            "epsilon_stabilizer": 0.001,
        }

    # ----- Node & Edge API ----- #
    def add_node(self, node: NodeData) -> None:
        self.graph.add_node(node.id, **asdict(node))

    def add_edge(self, edge: EdgeData) -> None:
        self.graph.add_edge(edge.source, edge.target, **asdict(edge))

    # ----- Metric Calculations ----- #
    def _contradiction_density(self, sub_nodes: List[str] | None = None) -> float:
        sub_edges = self.graph.edges(sub_nodes, data=True) if sub_nodes else
self.graph.edges(data=True)
        total_weight = sum(d["weight"] for *_edge, d in sub_edges)
        if total_weight == 0:
            return 0.0
        contradict_weight = sum(d["weight"] for *_edge, d in sub_edges if d.get("type") ==
"contradict")
        return contradict_weight / total_weight

    def _graph_entropy(self) -> float:
        """Simplistic entropy: degree distribution."""
        import math

```

```

degrees = [self.graph.degree(n) for n in self.graph.nodes]
total = sum(degrees)
if total == 0:
    return 0.0
probs = [d / total for d in degrees]
return -sum(p * math.log(p) for p in probs if p > 0)

def _coherence_mass(self) -> float:
    entropy = self._graph_entropy()
    eps = self.params["epsilon_stabilizer"]
    return 1 / (entropy + eps)

def _projection_bias(self) -> float:
    edges = self.graph.edges(data=True)
    total_weight = sum(d["weight"] for *_edge, d in edges)
    if total_weight == 0:
        return 0.0
    proj_weight = sum(d["weight"] for *_edge, d in edges if d.get("type") == "projects_to")
    return proj_weight / total_weight

# ----- Safety Checks ----- #
def _safety_breach(self) -> bool:
    if self._contradiction_density() > self.params["paradox_tolerance"]:
        return True
    if self._coherence_mass() < self.params["entropy_collapse_cutoff"]:
        return True
    if self._projection_bias() > self.params["projection_bias_alert"]:
        return True
    max_depth = max((d["recursion_depth"] for _, d in self.graph.nodes(data=True)), default=0)
    if max_depth > self.params["max_recursion_depth"]:
        return True
    overloaded = any(
        d["load_pressure"] > self.params["load_pressure_threshold"]
        for _, d in self.graph.nodes(data=True)
    )
    return overloaded

# ----- Snapshot / Rollback ----- #
def snapshot_state(self) -> None:
    snap = {
        "timestamp": datetime.utcnow().isoformat() + "Z",
        "nodes": {n: dict(d) for n, d in self.graph.nodes(data=True)},
        "edges": [(e[:2], dict(e[2])) for e in self.graph.edges(data=True)],
    }

```

```

self.snapshots.append(snap)

def rollback_last(self) -> None:
    if not self.snapshots:
        return
    last = self.snapshots.pop()
    self.graph.clear()
    self.graph.add_nodes_from([(n, d) for n, d in last["nodes"].items()])
    self.graph.add_edges_from([(s, t, d) for s, t, d in last["edges"]])

# ----- Mutative Operation Wrapper ----- #
def mutate(self, operation_fn, *args, **kwargs):
    """Wrap any graph-mutation in safety monitoring."""
    self.snapshot_state()
    operation_fn(*args, **kwargs)
    if self._safety_breach():
        if self.params["rollback_on_flag"]:
            self.rollback_last()
            raise RuntimeError("Safety breach detected — operation rolled back.")
        else:
            print("⚠ Safety breach detected; continuing by policy.")

# ----- Persistence ----- #
def save_json(self, path: str) -> None:
    data = {
        "nodes": [dict(n[1]) | {"id": n[0]} for n in self.graph.nodes(data=True)],
        "edges": [dict(d) | {"source": s, "target": t} for s, t, d in self.graph.edges(data=True)],
        "parameters": self.params,
    }
    with open(path, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2)

def load_json(self, path: str) -> None:
    with open(path, "r", encoding="utf-8") as f:
        data = json.load(f)
    self.graph.clear()
    for n in data["nodes"]:
        nid = n.pop("id")
        self.graph.add_node(nid, **n)
    for e in data["edges"]:
        src, tgt = e.pop("source"), e.pop("target")
        self.graph.add_edge(src, tgt, **e)
    self.params = data.get("parameters", self.params)

```

```
# =====
# Example Usage (for local testing only — strip in production)
# =====
if __name__ == "__main__":
    engine = JanusGraphEngine()

    # Build a trivial graph
    n1 = NodeData("node_001", "belief", 0.8, 0.9, 2, 0.1)
    n2 = NodeData("node_002", "memory", 0.5, 0.8, 1, 0.05)
    e1 = EdgeData("node_001", "node_002", 0.6, "projects_to")

    # Safe mutate wrapper
    engine.mutate(engine.add_node, n1)
    engine.mutate(engine.add_node, n2)
    engine.mutate(engine.add_edge, e1)

    print("Contradiction density:", engine._contradiction_density())
    print("Coherence mass:", engine._coherence_mass())
    print("Projection bias:", engine._projection_bias())

    engine.save_json("/tmp/janus_graph_snapshot.json")
```

Janus 5.0 — DOG/GOD Annotation Model

Spec Version 0.1 · 2025-07-01

This add-on spec formalizes how alignment metadata (DOG ↔ GOD) is stored, queried, and audited within the Janus 5.0 graph engine. Two interchangeable back-ends are defined:

Mode	Storage Location	Pros	Cons	Recommended When
Meta-Edge <i>(inline)</i>	<code>alignment</code> field inside each Edge / optional <code>alignment</code> on Node	• Single source of truth • No extra infrastructure • Simpler queries (<code>G[E]</code> filter)	• Slower to diff audit logs on large graphs • Coupling alignment & cognition may expose	• R&D prototypes • Graphs < 10k edges

			hazard labels	
Side-Channel I (parallel store)	Separate document collection (<code>alignment_store.jsonl</code>) keyed by (<code>source, target, timestamp</code>) or (<code>node_id, timestamp</code>)	• Fast time-series diffing • Easy to quarantine / strip sensitive fields • Scales to millions of edges	• Two-phase commit complexity • Slight storage overhead	• Production ops • Compliance or red-team audits

1 Meta-Edge Schema (Default)

```
// Edge example (inline alignment)
{
  "source": "node_007",
  "target": "node_042",
  "weight": 0.45,
  "type": "reflects_on",
  "alignment": "DOG" // or "GOD" or null
}

// Node example with self-alignment tag
{
  "id": "node_007",
  "type": "schema",
  "base_weight": 0.72,
  "stability_score": 0.62,
  "recursion_depth": 4,
  "load_pressure": 0.18,
  "alignment": "GOD"
}
```

1.1 Query Helpers

```
# return subgraph of DOG-aligned edges
DOG_view = G.edge_subgraph([(u,v) for u,v,d in G.edges(data=True) if d.get("alignment") == "DOG"])

# mutate: toggle alignment
engine.mutate(lambda: G[u][v].update({"alignment": "GOD"}))
```

2 Side-Channel Schema

Alignment events live in an **append-only** log:

```
{"timestamp": "2025-07-01T15:01:25Z", "source": "node_007", "target": "node_042", "alignment": "DOG"}
{"timestamp": "2025-07-01T15:02:03Z", "source": "node_042", "alignment": "GOD"}
```

2.1 Two-Phase Commit

- 1. **Stage** cognitive mutation on graph (no alignment).
- 2. **Commit** alignment record to `alignment_store`.
- 3. **Verify** both succeed → finalize snapshot (or rollback both).

2.2 API Hooks (engine extension)

```
engine.align_edge("node_007", "node_042", "DOG")
engine.align_node("node_042", "GOD")
log = engine.fetch_alignment_history("node_042")
```

3 Safety & Audit Rules

Rule	Enforcement
Alignment must be one of <code>DOG</code> , <code>GOD</code> , or <code>null</code> .	Engine validator.
No edge may hold both <code>DOG</code> and <code>GOD</code> simultaneously.	<code>validate_graph_integrity()</code> check.
Alignment flips require snapshot diff note.	Auto-logged by <code>mutate()</code> wrapper.
Side-channel file is append-only; no in-place edits.	File handler denies <code>w+</code> .

4 Choosing a Mode

graph TD

A[Proof-of-concept] -->|<10k edges| M(Meta-Edge)

A -->|Security/Compliance| S(Side-Channel)

B[Prod rollout] --> S

B --> M

Start with **Meta-Edge** (already supported in `graph_engine_stub.py`). If simulations crest ~10k edges or require external hazard audits, flip `config["alignment_mode"] = "side_channel"`.

5 Engine Extension Snippet

Add to `JanusGraphEngine.__init__()`:

```
self.alignment_mode = self.params.get("alignment_mode", "meta_edge")
if self.alignment_mode == "side_channel":
    self._alignment_store_path = self.params.get("alignment_store", "alignment_store.jsonl")
```

And new helper functions (`align_edge`, `align_node`, `fetch_alignment_history`) that branch on mode.

PR Note: full patched code coming once you confirm mode preference.

"""graph_engine.py

Janus 5.0 Core Graph Engine — Dual-Mode Alignment Build

Spec Version: 0.1 · 2025-07-01 (DOG/GOD model §2025-07-01)

Language : Python 3.12 + NetworkX ≥ 3.0

Implements **both** alignment storage modes:

1. **Meta-Edge** (inline) — default for <10 k edges / rapid prototyping.
2. **Side-Channel** (append-only JSONL log) — for large graphs & audits.

Toggle via `parameters["alignment_mode"] = "meta_edge" | "side_channel"`.

All canonical variables, metrics, and safety thresholds mirror
`janus_5_spec.md` and `dog_god_annotation_model.md`.

"""

from **future** import annotations

```
from dataclasses import dataclass, asdict
from typing import List, Dict, Any, Generator, Tuple
from datetime import datetime
import json, os, math
import networkx as nx
```

=====

Dataclass Definitions

=====

```
@dataclass
class NodeData:
    id: str
    type: str # belief|memory|schema|emotion|contradiction
    base_weight: float
    stability_score: float
    recursion_depth: int
    load_pressure: float
    labels: List[str] | None = None
    alignment: str | None = None # DOG | GOD | None
```

```
@dataclass
class EdgeData:
    source: str
    target: str
    weight: float
```

```
type: str # reinforce|contradict|blend|anchor|reflects_on|projects_to
timestamps: List[str] | None = None
alignment: str | None = None # DOG | GOD | None
```

=====

Helper Utilities

=====

```
def _now_iso() -> str:
    return datetime.utcnow().isoformat(timespec="seconds") + "Z"
```

=====

Engine Core

=====

```
class JanusGraphEngine:
    """Wraps an nx.DiGraph with Janus 5.0 cognitive-safety & alignment plumbing."""

    # ----- Init ----- #
    def __init__(self, parameters: Dict[str, Any] | None = None):
        self.graph = nx.DiGraph()
        self.snapshots: List[Dict[str, Any]] = []

    # Load/default parameters (spec §5)
    self.params = {
        "max_recursion_depth": 10,
        "paradox_tolerance": 0.35,
        "load_pressure_threshold": 0.65,
        "entropy_collapse_cutoff": 0.25,
        "projection_bias_alert": 0.8,
```

```

        "rollback_on_flag": True,
        "epsilon_stabilizer": 1e-3,
        "alignment_mode": "meta_edge",    # or "side_channel"
        "alignment_store": "alignment_store.jsonl",
    }
    if parameters:
        self.params.update(parameters)

    self.alignment_mode = self.params["alignment_mode"].lower()
    if self.alignment_mode not in {"meta_edge", "side_channel"}:
        raise ValueError("alignment_mode must be 'meta_edge' or 'side_channel'")

    # Ensure side-channel log exists
    if self.alignment_mode == "side_channel":
        path = self.params["alignment_store"]
        if not os.path.exists(path):
            open(path, "w", encoding="utf-8").close()

# ----- Node & Edge API ----- #
def add_node(self, node: NodeData) -> None:
    self.graph.add_node(node.id, **asdict(node))

def add_edge(self, edge: EdgeData) -> None:
    self.graph.add_edge(edge.source, edge.target, **asdict(edge))

# ----- Alignment Helpers ----- #
def align_node(self, node_id: str, alignment: str) -> None:
    alignment = alignment.upper()
    if alignment not in {"DOG", "GOD"}:
        raise ValueError("alignment must be 'DOG' or 'GOD'")

    def _op():
        if self.alignment_mode == "meta_edge":
            self.graph.nodes[node_id]["alignment"] = alignment
        else:
            self._append_alignment_log({
                "timestamp": _now_iso(),
                "node": node_id,
                "alignment": alignment,
            })
    self.mutate(_op)

def align_edge(self, source: str, target: str, alignment: str) -> None:
    alignment = alignment.upper()

```

```

if alignment not in {"DOG", "GOD"}:
    raise ValueError("alignment must be 'DOG' or 'GOD'")

def _op():
    if self.alignment_mode == "meta_edge":
        self.graph[source][target]["alignment"] = alignment
    else:
        self._append_alignment_log({
            "timestamp": _now_iso(),
            "source": source,
            "target": target,
            "alignment": alignment,
        })
    self.mutate(_op)

def fetch_alignment_history(self, entity_id: str) -> List[Dict[str, Any]]:
    """Returns side-channel records for a node or edge entity (side_channel mode).
    In meta_edge mode, returns current inline tag."""
    if self.alignment_mode == "meta_edge":
        # return live tag snapshot
        if entity_id in self.graph.nodes:
            return [{"current_alignment": self.graph.nodes[entity_id].get("alignment")}]
        return []
    # side-channel — scan log file
    records: List[Dict[str, Any]] = []
    path = self.params["alignment_store"]
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            if not line.strip():
                continue
            rec = json.loads(line)
            if rec.get("node") == entity_id or (
                rec.get("source") == entity_id or rec.get("target") == entity_id
            ):
                records.append(rec)
    return records

def _append_alignment_log(self, record: Dict[str, Any]):
    path = self.params["alignment_store"]
    with open(path, "a", encoding="utf-8") as f:
        json.dump(record, f)
        f.write("\n")

# ----- Metric Calculations ----- #

```

```

def _contradiction_density(self, nodes: List[str] | None = None) -> float:
    edges = self.graph.edges(nodes, data=True) if nodes else self.graph.edges(data=True)
    total = sum(d["weight"] for *_e, d in edges)
    if total == 0:
        return 0.0
    contradict = sum(d["weight"] for *_e, d in edges if d.get("type") == "contradict")
    return contradict / total

def _graph_entropy(self) -> float:
    degrees = [self.graph.degree(n) for n in self.graph.nodes]
    tot = sum(degrees)
    if tot == 0:
        return 0.0
    probs = [d / tot for d in degrees]
    return -sum(p * math.log(p) for p in probs if p > 0)

def _coherence_mass(self) -> float:
    entropy = self._graph_entropy()
    eps = self.params["epsilon_stabilizer"]
    return 1 / (entropy + eps)

def _projection_bias(self) -> float:
    edges = self.graph.edges(data=True)
    total = sum(d["weight"] for *_e, d in edges)
    if total == 0:
        return 0.0
    proj = sum(d["weight"] for *_e, d in edges if d.get("type") == "projects_to")
    return proj / total

# ----- Safety Checks ----- #
def _safety_breach(self) -> bool:
    if self._contradiction_density() > self.params["paradox_tolerance"]:
        return True
    if self._coherence_mass() < self.params["entropy_collapse_cutoff"]:
        return True
    if self._projection_bias() > self.params["projection_bias_alert"]:
        return True
    max_depth = max((d["recursion_depth"] for _, d in self.graph.nodes(data=True)), default=0)
    if max_depth > self.params["max_recursion_depth"]:
        return True
    overloaded = any(
        d["load_pressure"] > self.params["load_pressure_threshold"] for _, d in
        self.graph.nodes(data=True)
    )

```

return overloaded

----- Snapshot / Rollback -----

def snapshot_state(self) -> None:

```
    snap = {
        "timestamp": _now_iso(),
        "nodes": {n: dict(d) for n, d in self.graph.nodes(data=True)},
        "edges": [(e[2], dict(e[2])) for e in self.graph.edges(data=True)],
    }
    if self.alignment_mode == "side_channel":
        # include alignment file offset for accurate rollback
        path = self.params["alignment_store"]
        snap["alignment_log_size"] = os.path.getsize(path)
    self.snapshots.append(snap)
```

def rollback_last(self) -> None:

```
    if not self.snapshots:
        return
    last = self.snapshots.pop()
    # restore graph
    self.graph.clear()
    self.graph.add_nodes_from([(n, d) for n, d in last["nodes"].items()])
    self.graph.add_edges_from([(s, t, d) for s, t, d in last["edges"]])
    # truncate alignment log if needed
    if self.alignment_mode == "side_channel":
        path = self.params["alignment_store"]
        with open(path, "rb+") as f:
            f.truncate(last.get("alignment_log_size", 0))
```

----- Mutative Operation Wrapper -----

def mutate(self, op_fn, *args, **kwargs):

```
    """Wrap any graph-mutation in safety monitoring and atomic rollback."""
    self.snapshot_state()
    try:
        op_fn(*args, **kwargs)
        if self._safety_breach():
            raise RuntimeError("Safety breach detected after mutation.")
    except Exception as exc:
        if self.params["rollback_on_flag"]:
            self.rollback_last()
        raise exc
```

----- Persistence -----

def save_json(self, path: str) -> None:

```

data = {
    "nodes": [dict(n[1]) | {"id": n[0]} for n in self.graph.nodes(data=True)],
    "edges": [dict(d) | {"source": s, "target": t} for s, t, d in self.graph.edges(data=True)],
    "parameters": self.params,
}
with open(path, "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2)

def load_json(self, path: str) -> None:
    with open(path, "r", encoding="utf-8") as f:
        data = json.load(f)
    self.graph.clear()
    for n in data["nodes"]:
        nid = n.pop("id")
        self.graph.add_node(nid, **n)
    for e in data["edges"]:
        src, tgt = e.pop("source"), e.pop("target")
        self.graph.add_edge(src, tgt, **e)
    self.params.update(data.get("parameters", {}))
    self.alignment_mode = self.params["alignment_mode"].lower()

```

=====

Example CLI Use (dev-only)

=====

```

if name == "main":
    engine = JanusGraphEngine({"alignment_mode": "side_channel"})

    n1 = NodeData("n1", "belief", 0.9, 0.95, 2, 0.1)
    n2 = NodeData("n2", "memory", 0.4, 0.75, 1, 0.05)
    e1 = EdgeData("n1", "n2", 0.6, "projects_to")

    engine.mutate(engine.add_node, n1)
    engine.mutate(engine.add_node, n2)
    engine.mutate(engine.add_edge, e1)

```

```
engine.align_edge("n1", "n2", "DOG")
engine.align_node("n1", "GOD")
```

```
print("Alignment history n1:", engine.fetch_alignment_history("n1"))
print("Projection bias:", engine._projection_bias())
```

```
"""janus_module_adapters.py
```

```
-----
Adapters that wrap JanusCore 3.0 functional-psych modules so they can
operate directly on the Janus 5.0 Graph Engine.
```

Each adapter follows the same pattern:

- + Receives a reference to a `JanusGraphEngine` instance.
- + Exposes a `process(payload)` method (sync) that:
 1. Parses the payload (user text or session data)
 2. Derives symbolic outputs per module spec
 3. Applies those outputs as safe mutations on the graph via
`engine.mutate()` (adds nodes/edges, sets attributes, or
alignment tags).
- + Returns a structured dict with:
{"graph_delta": <brief summary>, "module_output": <raw block>}

This keeps the JSON API consistent for higher-level orchestration shells.

NOTE: Real NLP / analysis logic is stubbed as TODO. The goal is to show how each module *hooks* into the Graph Engine, leaving the heavy lifting open to later plug-ins (regex, spaCy, LLM calls, etc.).

```
"""
```

```
from __future__ import annotations
```

```
from abc import ABC, abstractmethod
from typing import Any, Dict, List
import uuid
```

```
from graph_engine import JanusGraphEngine, NodeData, EdgeData
```

```
# =====
# Base Adapter Template
# =====
class BaseModuleAdapter(ABC):
    module_name: str = "BaseAdapter"

    def __init__(self, engine: JanusGraphEngine):
```

```

self.engine = engine

@abstractmethod
def process(self, payload: Any) -> Dict[str, Any]:
    """Runs the module logic and mutates the graph in a safe wrapper."""

# Helper: commit node safely
def _add_node(self, node: NodeData):
    self.engine.mutate(self.engine.add_node, node)

# Helper: commit edge safely
def _add_edge(self, edge: EdgeData):
    self.engine.mutate(self.engine.add_edge, edge)

# =====
# SchemaMapper Adapter
# =====
class SchemaMapperAdapter(BaseModuleAdapter):
    module_name = "SchemaMapper"

    def process(self, text: str) -> Dict[str, Any]:
        # TODO: real schema detection; stub below
        detected_schemas = ["abandonment", "failure"]
        confidence = 0.84
        trace_id = f"trace_schema_{uuid.uuid4().hex[:6]}"

        # Create node for schema profile
        node_id = f"schema_profile_{trace_id}"
        schema_node = NodeData(
            id=node_id,
            type="schema",
            base_weight=0.6,
            stability_score=0.7,
            recursion_depth=1,
            load_pressure=0.2,
            labels=detected_schemas,
        )
        self._add_node(schema_node)

        # Edge from profile to text stub node (simulate linkage)
        text_node_id = f"text_{uuid.uuid4().hex[:6]}"
        text_node = NodeData(text_node_id, "memory", 0.3, 0.6, 0, 0.1, labels=["input"])
        self._add_node(text_node)

```

```

link_edge = EdgeData(text_node_id, node_id, 0.7, "reflects_on")
self._add_edge(link_edge)

```

```

return {
    "graph_delta": {
        "nodes_added": [node_id, text_node_id],
        "edges_added": [(text_node_id, node_id)],
    },
    "module_output": {
        "dominant_schemas": detected_schemas,
        "confidence": confidence,
        "trace_id": trace_id,
    },
}

```

```

# =====

```

```

# PersonaSimulator Adapter

```

```

# =====

```

```

class PersonaSimulatorAdapter(BaseModuleAdapter):

```

```

    module_name = "PersonaSimulator"

```

```

    def process(self, scenario: str) -> Dict[str, Any]:

```

```

        # TODO: real mask reasoning; stub outputs

```

```

        masks = {

```

```

            "Guide": "This path has long-term benefits.",

```

```

            "Challenger": "It's risky but exciting.",

```

```

            "Mediator": "Try a limited experiment first.",

```

```

        }

```

```

        arbiter = "CERBERUS: Consensus — conditional risk." # stub

```

```

        conflict_intensity = "moderate"

```

```

        sim_node_id = f"persona_sim_{uuid.uuid4().hex[:6]}"

```

```

        sim_node = NodeData(

```

```

            sim_node_id,

```

```

            "emotion",

```

```

            base_weight=0.4,

```

```

            stability_score=0.5,

```

```

            recursion_depth=2,

```

```

            load_pressure=0.3,

```

```

            labels=["persona_simulation"],

```

```

        )

```

```

        self._add_node(sim_node)

```

```

    # Attach mask edges

```

```

for mask, utter in masks.items():
    mask_node_id = f"mask_{mask.lower()}"
    if mask_node_id not in self.engine.graph:
        self._add_node(NodeData(mask_node_id, "schema", 0.5, 0.8, 1, 0.15,
labels=[mask]))
        edge = EdgeData(mask_node_id, sim_node_id, 0.5, "projects_to")
        self._add_edge(edge)

    return {
        "graph_delta": {"nodes_added": [sim_node_id], "edges_added": "mask_links"},
        "module_output": {
            "masks": masks,
            "arbiter": arbiter,
            "conflict_intensity": conflict_intensity,
        },
    }

# =====
# BiasScanner Adapter
# =====
class BiasScannerAdapter(BaseModuleAdapter):
    module_name = "BiasScanner"

    def process(self, text: str) -> Dict[str, Any]:
        # TODO: real bias detection
        biases = [
            {
                "bias": "sunk_cost",
                "trigger": "I can't quit now—spent too much time.",
                "suggested_reframe": "Would you still choose this if starting fresh?",
            }
        ]
        report_node_id = f"bias_report_{uuid.uuid4().hex[:6]}"
        report_node = NodeData(report_node_id, "contradiction", 0.3, 0.4, 1, 0.25, labels=["bias"])
        self._add_node(report_node)

        # Edge from report to text node (simulate)
        text_node_id = f"text_{uuid.uuid4().hex[:6]}"
        self._add_node(NodeData(text_node_id, "memory", 0.2, 0.6, 0, 0.1, labels=["input"]))
        self._add_edge(EdgeData(text_node_id, report_node_id, 0.6, "contradict"))

    return {
        "graph_delta": {"nodes_added": [report_node_id, text_node_id]},
        "module_output": {"biases": biases},
    }

```

```

    }

# =====
# ReportBuilder Adapter (prototype)
# =====
class ReportBuilderAdapter(BaseModuleAdapter):
    module_name = "ReportBuilder"

    def process(self, _: Any = None) -> Dict[str, Any]:
        # Simple illustration: pull some metrics & draft markdown
        metrics = {
            "contradiction_density": round(self.engine._contradiction_density(), 3),
            "coherence_mass": round(self.engine._coherence_mass(), 3),
            "projection_bias": round(self.engine._projection_bias(), 3),
        }
        md_report = (
            "## JANUS SESSION REPORT\n"
            f"*Contradiction Density*: {metrics['contradiction_density']}\n\n"
            f"*Coherence Mass*: {metrics['coherence_mass']}\n\n"
            f"*Projection Bias*: {metrics['projection_bias']}\n\n"
        )
        # Node to store report snapshot
        report_node_id = f"report_{uuid.uuid4().hex[:6]}"
        self._add_node(NodeData(report_node_id, "memory", 0.1, 0.9, 0, 0.05, labels=["report"]))
        return {"graph_delta": {"nodes_added": [report_node_id]}, "module_output": md_report}

# =====
# CodexExporter Adapter (prototype)
# =====
class CodexExporterAdapter(BaseModuleAdapter):
    module_name = "CodexExporter"

    def process(self, _: Any = None) -> Dict[str, Any]:
        # Serialize core graph (placeholder compression)
        codex = {
            "nodes": len(self.engine.graph.nodes),
            "edges": len(self.engine.graph.edges),
            "entropy": round(self.engine._graph_entropy(), 3),
        }
        return {"graph_delta": {}, "module_output": codex}

# =====
# Adapter Registry Helper
# =====

```

```

ADAPTER_REGISTRY = {
    "SchemaMapper": SchemaMapperAdapter,
    "PersonaSimulator": PersonaSimulatorAdapter,
    "BiasScanner": BiasScannerAdapter,
    "ReportBuilder": ReportBuilderAdapter,
    "CodexExporter": CodexExporterAdapter,
}

```

```

def get_adapter(name: str, engine: JanusGraphEngine) -> BaseModuleAdapter:
    cls = ADAPTER_REGISTRY.get(name)
    if not cls:
        raise ValueError(f"Adapter '{name}' not registered.")
    return cls(engine)

```

```

# =====
# Example Workflow (dev-only)
# =====

```

```

if __name__ == "__main__":
    from graph_engine import JanusGraphEngine

```

```

    eng = JanusGraphEngine()
    schema_adapter = get_adapter("SchemaMapper", eng)
    persona_adapter = get_adapter("PersonaSimulator", eng)
    bias_adapter = get_adapter("BiasScanner", eng)
    report_adapter = get_adapter("ReportBuilder", eng)

```

```

    schema_adapter.process("I often feel left out and fear failure.")
    persona_adapter.process("Should I quit my job to start a company?")
    bias_adapter.process("I can't quit now—spent too much time.")
    summary = report_adapter.process()

```

```

    print(summary["module_output"])

```

```

import { useState } from "react";
import { Card, CardContent } from "@components/ui/card";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Textarea } from "@components/ui/textarea";
import { motion } from "framer-motion";
import { LineChart, Line, XAxis, YAxis, Tooltip, ResponsiveContainer } from "recharts";

```

```

/**

```

```

 * JanusDashboard.jsx

```

```

 * A minimal operator UI for Janus 5.0 that wraps common module actions

```

* and visualizes core metrics.

*

* NOTE: Back-end calls are stubbed — replace `fakeEngineCall` with real

* IPC / WebSocket hooks to your JanusGraphEngine process.

*/

```
export default function JanusDashboard() {
  const [input, setInput] = useState("");
  const [log, setLog] = useState([]);
  const [metrics, setMetrics] = useState({
    contradiction_density: 0,
    coherence_mass: 0,
    projection_bias: 0,
  });

  const handleRun = async (adapter) => {
    if (!input.trim()) return;
    // Stubbed engine call
    const result = await fakeEngineCall(adapter, input);
    setLog((prev) => [result.module_output, ...prev]);
    setMetrics(result.metrics);
    setInput("");
  };

  return (
    <div className="grid md:grid-cols-3 gap-4 p-4">
      {/* Interaction Panel */}
      <motion.div
        initial={{ x: -50, opacity: 0 }}
        animate={{ x: 0, opacity: 1 }}
        className="md:col-span-1 space-y-4"
      >
        <Card className="p-4 space-y-2">
          <h2 className="text-xl font-bold">Operator Console</h2>
          <Textarea
            placeholder="Enter text, scenario, or question..."
            value={input}
            onChange={(e) => setInput(e.target.value)}
            className="min-h-[120px]"
          />
          <div className="grid grid-cols-2 gap-2">
            <Button onClick={() => handleRun("SchemaMapper")}>Schema Map</Button>
            <Button onClick={() => handleRun("PersonaSimulator")}>Sim Personas</Button>
            <Button onClick={() => handleRun("BiasScanner")} variant="secondary">
```

```

        Scan Bias
      </Button>
      <Button onClick={() => handleRun("ReportBuilder")} variant="secondary">
        Build Report
      </Button>
    </div>
  </Card>

  <Card className="p-4 overflow-y-auto max-h-[40vh]">
    <h3 className="font-semibold mb-2">Session Log</h3>
    {log.length === 0 && <p className="text-sm text-muted">No events yet.</p>}
    {log.map((entry, i) => (
      <pre key={i} className="text-xs whitespace-pre-wrap mb-2">
        {JSON.stringify(entry, null, 2)}
      </pre>
    ))}
  </Card>
</motion.div>

{/* Metrics Panel */}
<motion.div
  initial={{ x: 50, opacity: 0 }}
  animate={{ x: 0, opacity: 1 }}
  className="md:col-span-2 space-y-4"
>
  <Card className="p-4">
    <h2 className="text-xl font-bold mb-4">Real-Time Metrics</h2>
    <div className="grid grid-cols-3 gap-4 text-center">
      <MetricBox label="Contradiction Density" value={metrics.contradiction_density} />
      <MetricBox label="Coherence Mass" value={metrics.coherence_mass} />
      <MetricBox label="Projection Bias" value={metrics.projection_bias} />
    </div>
  </Card>

  <Card className="p-4">
    <h2 className="text-xl font-bold mb-4">Entropy Trend</h2>
    <ResponsiveContainer width="100%" height={200}>
      <LineChart data={entropySeries(log)}>
        <XAxis dataKey="index" hide />
        <YAxis domain={[0, "dataMax"]} />
        <Tooltip />
        <Line type="monotone" dataKey="entropy" strokeWidth={2} dot={false} />
      </LineChart>
    </ResponsiveContainer>
  </Card>

```

```

        </Card>
      </motion.div>
    </div>
  );
}

function MetricBox({ label, value }) {
  return (
    <div className="rounded-2xl shadow p-4">
      <p className="text-sm text-muted mb-1">{label}</p>
      <p className="text-2xl font-semibold" title={value}>{value.toFixed(3)}</p>
    </div>
  );
}

```

```

// ----- stub helpers ----- //
async function fakeEngineCall(adapter, input) {
  // simulate latency
  await new Promise((r) => setTimeout(r, 400));
  // trivial fake metrics trend
  const rand = () => Math.random() * 0.5;
  return {
    module_output: { adapter, input, timestamp: Date.now() },
    metrics: {
      contradiction_density: rand(),
      coherence_mass: rand() + 0.5,
      projection_bias: rand(),
    },
  };
}

```

```

function entropySeries(log) {
  return log.map((_, i) => ({ index: i, entropy: Math.random() + 0.5 }));
}

import { useState } from "react";
import { Card, CardContent } from "@components/ui/card";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Textarea } from "@components/ui/textarea";
import { motion } from "framer-motion";
import { LineChart, Line, XAxis, YAxis, Tooltip, ResponsiveContainer } from "recharts";

/**
 * JanusDashboard.jsx

```

```

* A minimal operator UI for Janus 5.0 that wraps common module actions
* and visualizes core metrics.
*
* NOTE: Back-end calls are stubbed — replace `fakeEngineCall` with real
* IPC / WebSocket hooks to your JanusGraphEngine process.
*/

```

```

export default function JanusDashboard() {
  const [input, setInput] = useState("");
  const [log, setLog] = useState([]);
  const [metrics, setMetrics] = useState({
    contradiction_density: 0,
    coherence_mass: 0,
    projection_bias: 0,
  });

  const handleRun = async (adapter) => {
    if (!input.trim()) return;
    // Stubbed engine call
    const result = await fakeEngineCall(adapter, input);
    setLog((prev) => [result.module_output, ...prev]);
    setMetrics(result.metrics);
    setInput("");
  };

  return (
    <div className="grid md:grid-cols-3 gap-4 p-4">
      { /* Interaction Panel */ }
      <motion.div
        initial={{ x: -50, opacity: 0 }}
        animate={{ x: 0, opacity: 1 }}
        className="md:col-span-1 space-y-4"
      >
        <Card className="p-4 space-y-2">
          <h2 className="text-xl font-bold">Operator Console</h2>
          <Textarea
            placeholder="Enter text, scenario, or question..."
            value={input}
            onChange={(e) => setInput(e.target.value)}
            className="min-h-[120px]"
          />
          <div className="grid grid-cols-2 gap-2">
            <Button onClick={() => handleRun("SchemaMapper")}>Schema Map</Button>
            <Button onClick={() => handleRun("PersonaSimulator")}>Sim Personas</Button>
          </div>
        </Card>
      </motion.div>
    </div>
  );
}

```

```

    <Button onClick={() => handleRun("BiasScanner")} variant="secondary">
      Scan Bias
    </Button>
    <Button onClick={() => handleRun("ReportBuilder")} variant="secondary">
      Build Report
    </Button>
  </div>
</Card>

```

```

<Card className="p-4 overflow-y-auto max-h-[40vh]">
  <h3 className="font-semibold mb-2">Session Log</h3>
  {log.length === 0 && <p className="text-sm text-muted">No events yet.</p>}
  {log.map((entry, i) => (
    <pre key={i} className="text-xs whitespace-pre-wrap mb-2">
      {JSON.stringify(entry, null, 2)}
    </pre>
  ))}
</Card>
</motion.div>

```

```

{/* Metrics Panel */}
<motion.div
  initial={{ x: 50, opacity: 0 }}
  animate={{ x: 0, opacity: 1 }}
  className="md:col-span-2 space-y-4"
>
  <Card className="p-4">
    <h2 className="text-xl font-bold mb-4">Real-Time Metrics</h2>
    <div className="grid grid-cols-3 gap-4 text-center">
      <MetricBox label="Contradiction Density" value={metrics.contradiction_density} />
      <MetricBox label="Coherence Mass" value={metrics.coherence_mass} />
      <MetricBox label="Projection Bias" value={metrics.projection_bias} />
    </div>
  </Card>

```

```

<Card className="p-4">
  <h2 className="text-xl font-bold mb-4">Entropy Trend</h2>
  <ResponsiveContainer width="100%" height={200}>
    <LineChart data={entropySeries(log)}>
      <XAxis dataKey="index" hide />
      <YAxis domain={[0, "dataMax"]} />
      <Tooltip />
      <Line type="monotone" dataKey="entropy" strokeWidth={2} dot={false} />
    </LineChart>

```

```

        </ResponsiveContainer>
      </Card>
    </motion.div>
  </div>
);
}

function MetricBox({ label, value }) {
  return (
    <div className="rounded-2xl shadow p-4">
      <p className="text-sm text-muted mb-1">{label}</p>
      <p className="text-2xl font-semibold" title={value}>{value.toFixed(3)}</p>
    </div>
  );
}

```

```

// ----- stub helpers ----- //
async function fakeEngineCall(adapter, input) {
  // simulate latency
  await new Promise((r) => setTimeout(r, 400));
  // trivial fake metrics trend
  const rand = () => Math.random() * 0.5;
  return {
    module_output: { adapter, input, timestamp: Date.now() },
    metrics: {
      contradiction_density: rand(),
      coherence_mass: rand() + 0.5,
      projection_bias: rand(),
    },
  };
}

```

```

function entropySeries(log) {
  return log.map((_, i) => ({ index: i, entropy: Math.random() + 0.5 }));
}

```

```

import { useState } from "react";
import { Card } from "@components/ui/card";
import { Button } from "@components/ui/button";
import { Textarea } from "@components/ui/textarea";
import { motion } from "framer-motion";
import { LineChart, Line, XAxis, YAxis, Tooltip, ResponsiveContainer } from "recharts";
import Joyride, { STATUS } from "react-joyride";

```

```

/**
 * JanusDashboard.jsx ▶ Operator UI + Guided Tour
 * -----
 * Minimal dashboard for Janus 5.0 with:
 * • Ritual buttons → module adapters
 * • Real-time metrics cards + entropy chart
 * • Guided onboarding overlay (react-joyride)
 *
 * Replace `fakeEngineCall` with real IPC / API hooks.
 */

export default function JanusDashboard() {
  const [input, setInput] = useState("");
  const [log, setLog] = useState([]);
  const [metrics, setMetrics] = useState({
    contradiction_density: 0,
    coherence_mass: 0,
    projection_bias: 0,
  });

  /* ----- Guided Tour State ----- */
  const [tourRun, setTourRun] = useState(true);
  const [tourStepIndex, setTourStepIndex] = useState(0);
  const tourSteps = [
    {
      target: "#input-pane",
      content:
        "Welcome to Janus! Type a concern or scenario here, then choose an action to begin the ritual.",
      disableBeacon: true,
    },
    {
      target: "#action-buttons",
      content:
        "Each button invokes a Janus module: map schemas, simulate personas, scan biases, or compile a report.",
    },
    {
      target: "#session-log",
      content: "Module outputs log here so you can audit each step.",
    },
    {
      target: "#metrics-panel",
      content:

```

```
    "Live metrics gauge system stability. Keep contradiction density low and coherence mass high!",
```

```
  },  
  {  
    target: "#entropy-chart",  
    content: "Entropy trend visualises systemic disorder over time.",  
  },  
];
```

```
const handleTourCallback = ({ status, index }) => {  
  setTourStepIndex(index);  
  if ([STATUS.FINISHED, STATUS.SKIPPED].includes(status)) setTourRun(false);  
};
```

```
/* _____ Adapter Trigger _____ */  
const handleRun = async (adapter) => {  
  if (!input.trim()) return;  
  const result = await fakeEngineCall(adapter, input);  
  setLog((prev) => [result.module_output, ...prev]);  
  setMetrics(result.metrics);  
  setInput("");  
};
```

```
return (  
  <div className="grid md:grid-cols-3 gap-4 p-4 relative">  
    { /* Guided Tour Overlay */ }  
    <Joyride  
      steps={tourSteps}  
      run={tourRun}  
      stepIndex={tourStepIndex}  
      continuous  
      showSkipButton  
      callback={handleTourCallback}  
      styles={{ options: { zIndex: 10000 } }}  
    />  
  
    { /* Interaction Panel */ }  
    <motion.div  
      initial={{ x: -50, opacity: 0 }}  
      animate={{ x: 0, opacity: 1 }}  
      className="md:col-span-1 space-y-4"  
    >  
      <Card id="input-pane" className="p-4 space-y-2">  
        <h2 className="text-xl font-bold">Operator Console</h2>
```

```

<Textarea
  placeholder="Enter text, scenario, or question..."
  value={input}
  onChange={(e) => setInput(e.target.value)}
  className="min-h-[120px]"
/>
<div id="action-buttons" className="grid grid-cols-2 gap-2">
  <Button onClick={() => handleRun("SchemaMapper")}>Schema Map</Button>
  <Button onClick={() => handleRun("PersonaSimulator")}>Sim Personas</Button>
  <Button onClick={() => handleRun("BiasScanner")} variant="secondary">
    Scan Bias
  </Button>
  <Button onClick={() => handleRun("ReportBuilder")} variant="secondary">
    Build Report
  </Button>
</div>
</Card>

<Card id="session-log" className="p-4 overflow-y-auto max-h-[40vh]">
  <h3 className="font-semibold mb-2">Session Log</h3>
  {log.length === 0 && <p className="text-sm text-muted">No events yet.</p>}
  {log.map((entry, i) => (
    <pre key={i} className="text-xs whitespace-pre-wrap mb-2">
      {JSON.stringify(entry, null, 2)}
    </pre>
  ))}
</Card>
</motion.div>

{/* Metrics Panel */}
<motion.div
  initial={{ x: 50, opacity: 0 }}
  animate={{ x: 0, opacity: 1 }}
  className="md:col-span-2 space-y-4"
>
  <Card id="metrics-panel" className="p-4">
    <h2 className="text-xl font-bold mb-4">Real-Time Metrics</h2>
    <div className="grid grid-cols-3 gap-4 text-center">
      <MetricBox label="Contradiction Density" value={metrics.contradiction_density} />
      <MetricBox label="Coherence Mass" value={metrics.coherence_mass} />
      <MetricBox label="Projection Bias" value={metrics.projection_bias} />
    </div>
  </Card>

```

```

<Card id="entropy-chart" className="p-4">
  <h2 className="text-xl font-bold mb-4">Entropy Trend</h2>
  <ResponsiveContainer width="100%" height={200}>
    <LineChart data={entropySeries(log)}>
      <XAxis dataKey="index" hide />
      <YAxis domain={[0, "dataMax"]} />
      <Tooltip />
      <Line type="monotone" dataKey="entropy" strokeWidth={2} dot={false} />
    </LineChart>
  </ResponsiveContainer>
</Card>
</motion.div>

```

```

{/* Restart Tour Button */}
{!tourRun && (
  <Button
    onClick={() => {
      setTourRun(true);
      setTourStepIndex(0);
    }}
    className="fixed bottom-4 right-4 shadow-lg"
  >
    Restart Tour
  </Button>
)}
</div>
);
}

```

```

function MetricBox({ label, value }) {
  return (
    <div className="rounded-2xl shadow p-4">
      <p className="text-sm text-muted mb-1">{label}</p>
      <p className="text-2xl font-semibold" title={value}>{value.toFixed(3)}</p>
    </div>
  );
}

```

```

// ----- stub helpers ----- //
async function fakeEngineCall(adapter, input) {
  await new Promise((r) => setTimeout(r, 400));
  const rand = () => Math.random() * 0.5;
  return {
    module_output: { adapter, input, timestamp: Date.now() },

```

```

    metrics: {
      contradiction_density: rand(),
      coherence_mass: rand() + 0.5,
      projection_bias: rand(),
    },
  };
}

```

```

function entropySeries(log) {
  return log.map((_, i) => ({ index: i, entropy: Math.random() + 0.5 }));
}

```

```

import React, { useState, useEffect } from "react";
import ReactDOM from "react-dom/client";
import { Card } from "@components/ui/card";
import { Button } from "@components/ui/button";
import { Textarea } from "@components/ui/textarea";
import { motion } from "framer-motion";
import { LineChart, Line, XAxis, YAxis, Tooltip, ResponsiveContainer } from "recharts";
import Joyride, { STATUS } from "react-joyride";

```

```

/**
 * JanusDashboard.jsx ► Operator UI + Guided Tour + Auto-Mount
 * -----
 * Minimal dashboard for Janus 5.0 with:
 * • Ritual buttons → module adapters
 * • Real-time metrics cards + entropy chart
 * • Guided onboarding overlay (react-joyride)
 * • Auto-deploy — when this module is imported anywhere and the
 *   DOM contains a <div id="janus-root" />, it mounts itself.
 *
 * Swap `fakeEngineCall` with real IPC / API hooks.
 */

```

```

export default function JanusDashboard() {
  const [input, setInput] = useState("");
  const [log, setLog] = useState([]);
  const [metrics, setMetrics] = useState({
    contradiction_density: 0,
    coherence_mass: 0,
    projection_bias: 0,
  });
}

```

```

/* ----- Guided Tour State ----- */

```

```

const [tourRun, setTourRun] = useState(true);
const [tourStepIndex, setTourStepIndex] = useState(0);
const tourSteps = [
  {
    target: "#input-pane",
    content:
      "Welcome to Janus! Type a concern or scenario, then choose a ritual action.",
    disableBeacon: true,
  },
  {
    target: "#action-buttons",
    content:
      "Each button invokes a Janus module: map schemas, simulate personas, scan biases, or
build a report.",
  },
  { target: "#session-log", content: "Module outputs appear here for audit." },
  {
    target: "#metrics-panel",
    content: "Live metrics reflect system stability. Aim for high coherence mass!",
  },
  { target: "#entropy-chart", content: "Entropy trend shows systemic disorder over time." },
];

const handleTourCallback = ({ status, index }) => {
  setTourStepIndex(index);
  if ([STATUS.FINISHED, STATUS.SKIPPED].includes(status)) setTourRun(false);
};

/* _____ Adapter Trigger _____ */
const handleRun = async (adapter) => {
  if (!input.trim()) return;
  const result = await fakeEngineCall(adapter, input);
  setLog((prev) => [result.module_output, ...prev]);
  setMetrics(result.metrics);
  setInput("");
};

return (
  <div className="grid md:grid-cols-3 gap-4 p-4 relative">
    { /* Guided Tour */ }
    <Joyride
      steps={tourSteps}
      run={tourRun}
      stepIndex={tourStepIndex}
    />
  </div>
)

```

```

continuous
showSkipButton
callback={handleTourCallback}
styles={{ options: { zIndex: 10000 } }}
/>

{/* Interaction Panel */}
<motion.div
  initial={{ x: -50, opacity: 0 }}
  animate={{ x: 0, opacity: 1 }}
  className="md:col-span-1 space-y-4"
>
  <Card id="input-pane" className="p-4 space-y-2">
    <h2 className="text-xl font-bold">Operator Console</h2>
    <Textarea
      placeholder="Enter text, scenario, or question..."
      value={input}
      onChange={(e) => setInput(e.target.value)}
      className="min-h-[120px]"
    />
    <div id="action-buttons" className="grid grid-cols-2 gap-2">
      <Button onClick={() => handleRun("SchemaMapper")}>Schema Map</Button>
      <Button onClick={() => handleRun("PersonaSimulator")}>Sim Personas</Button>
      <Button onClick={() => handleRun("BiasScanner")}>Scan Bias</Button>
      <Button onClick={() => handleRun("ReportBuilder")} variant="secondary">
        Build Report
      </Button>
    </div>
  </Card>

  <Card id="session-log" className="p-4 overflow-y-auto max-h-[40vh]">
    <h3 className="font-semibold mb-2">Session Log</h3>
    {log.length === 0 ? (
      <p className="text-sm text-muted">No events yet.</p>
    ) : (
      log.map((entry, i) => (
        <pre key={i} className="text-xs whitespace-pre-wrap mb-2">
          {JSON.stringify(entry, null, 2)}
        </pre>
      ))
    )}
  </Card>
</motion.div>

```

```

    { /* Metrics Panel */
    <motion.div
      initial={{ x: 50, opacity: 0 }}
      animate={{ x: 0, opacity: 1 }}
      className="md:col-span-2 space-y-4"
    >
      <Card id="metrics-panel" className="p-4">
        <h2 className="text-xl font-bold mb-4">Real-Time Metrics</h2>
        <div className="grid grid-cols-3 gap-4 text-center">
          <MetricBox label="Contradiction Density" value={metrics.contradiction_density} />
          <MetricBox label="Coherence Mass" value={metrics.coherence_mass} />
          <MetricBox label="Projection Bias" value={metrics.projection_bias} />
        </div>
      </Card>

      <Card id="entropy-chart" className="p-4">
        <h2 className="text-xl font-bold mb-4">Entropy Trend</h2>
        <ResponsiveContainer width="100%" height={200}>
          <LineChart data={entropySeries(log)}>
            <XAxis dataKey="index" hide />
            <YAxis domain={[0, "dataMax"]} />
            <Tooltip />
            <Line type="monotone" dataKey="entropy" strokeWidth={2} dot={false} />
          </LineChart>
        </ResponsiveContainer>
      </Card>
    </motion.div>

    { /* Restart Tour */
    {!tourRun && (
      <Button
        onClick={() => {
          setTourRun(true);
          setTourStepIndex(0);
        }}
        className="fixed bottom-4 right-4 shadow-lg"
      >
        Restart Tour
      </Button>
    )}
  </div>
);
}

```

```

function MetricBox({ label, value }) {
  return (
    <div className="rounded-2xl shadow p-4">
      <p className="text-sm text-muted mb-1">{label}</p>
      <p className="text-2xl font-semibold" title={value}>{value.toFixed(3)}</p>
    </div>
  );
}

/* ----- Stub Helpers (replace) ----- */
async function fakeEngineCall(adapter, input) {
  await new Promise((r) => setTimeout(r, 400));
  const rand = () => Math.random() * 0.5;
  return {
    module_output: { adapter, input, timestamp: Date.now() },
    metrics: {
      contradiction_density: rand(),
      coherence_mass: rand() + 0.5,
      projection_bias: rand(),
    },
  };
}

function entropySeries(log) {
  return log.map((_, i) => ({ index: i, entropy: Math.random() + 0.5 }));
}

/* ----- Auto-Mount when Context Present ----- */
if (typeof window !== "undefined" && !window.__JANUS_DASHBOARD_MOUNTED__) {
  const mountPoint = document.getElementById("janus-root");
  if (mountPoint) {
    window.__JANUS_DASHBOARD_MOUNTED__ = true;
    const root = ReactDOM.createRoot(mountPoint);
    root.render(
      <React.StrictMode>
        <JanusDashboard />
      </React.StrictMode>
    );
  }
}

```